

实 验 报 告

实验名称: 实验一 Hopfield 模型的实现

课程名称: 认知科学与类脑计算

实验地点: K2 227

实验日期: 2025/9/12

班 级: 23 级智能班

姓 名: 宋浩宇

学 号: 202300130183

一 实验目的：

加深对 Hopfield 模型的理解，能够使用 Hopfield 模型解决实际问题

二 实验环境：

软件环境：

主系统：Windows 11 家庭中文版 23H2 ; ArchLinux

编辑器：Visual Studio Code ; PyCharm 2025.1.2 ; neovim 0.11

Python 解释器：Python 3.9

Pytorch 版本：2.8.0

CUDA 版本：12.8.97

硬件环境：

CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz

内存：32.0 GB (31.6 GB 可用)

磁盘驱动器：NVMe WD_BLACKSN850X2000GB

显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU

CUDA 核心数：7424

三 实验内容：

根据 Hopfield 神经网络的相关知识，设计一个具有联想记忆功能的

离散型 Hopfield 神经网络。要求该网络可以正确识别 0-9 这 10 个数字，当数字被一定的噪声干扰后，仍具有较好的识别效果。

四 实验步骤：

1. 实验程序：

首先是第一项设计数字点阵图，生成点阵图使用的代码如下：

```
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
cmap = mcolors.ListedColormap(['white', 'black'])
bounds = [-1.5, 0, 1.5]
norm = mcolors.BoundaryNorm(bounds, cmap.N)
# 数字点阵
import numpy as np
one = np.array([[ -1,  -1,  1,  -1,  -1],
                 [ -1,   1,  1,  -1,  -1],
                 [ -1,  -1,  1,  -1,  -1],
                 [ -1,  -1,  1,  -1,  -1],
                 [ -1,  -1,  1,  -1,  -1],
                 [ -1,   1,  1,   1,  -1]])
two = np.array([[ 1,  1,  1,  1,  1],
                 [-1, -1, -1, -1,  1],
                 [-1, -1, -1, -1,  1],
                 [ 1,  1,  1,  1,  1],
                 [ 1, -1, -1, -1, -1],
                 [ 1,  1,  1,  1,  1]])
three = np.array([[ 1,  1,  1,  1,  1],
                  [-1, -1, -1, -1,  1],
                  [ 1,  1,  1,  1,  1],
                  [-1, -1, -1, -1,  1],
                  [-1, -1, -1, -1,  1],
                  [ 1,  1,  1,  1,  1]])
four = np.array([[ 1, -1, -1, -1,  1],
                  [ 1, -1, -1, -1,  1],
                  [ 1, -1, -1, -1,  1],
                  [ 1,  1,  1,  1,  1],
                  [-1, -1, -1, -1,  1],
```

```

        [-1, -1, -1, -1, 1]])
five = np.array([[1, 1, 1, 1, 1],
                 [1, -1, -1, -1, -1],
                 [1, 1, 1, 1, 1],
                 [-1, -1, -1, -1, 1],
                 [-1, -1, -1, -1, 1],
                 [1, 1, 1, 1, 1]])
six = np.array([[1, 1, 1, 1, 1],
                [1, -1, -1, -1, -1],
                [1, 1, 1, 1, 1],
                [1, -1, -1, -1, 1],
                [1, -1, -1, -1, 1],
                [1, 1, 1, 1, 1]])
seven = np.array([[1, 1, 1, 1, 1],
                  [-1, -1, -1, -1, 1],
                  [-1, -1, -1, 1, -1],
                  [-1, -1, 1, -1, -1],
                  [-1, 1, -1, -1, -1],
                  [1, -1, -1, -1, -1]])
eight = np.array([[1, 1, 1, 1, 1],
                  [1, -1, -1, -1, 1],
                  [1, 1, 1, 1, 1],
                  [1, -1, -1, -1, 1],
                  [1, -1, -1, -1, 1],
                  [1, 1, 1, 1, 1]])
nine = np.array([[1, 1, 1, 1, 1],
                 [1, -1, -1, -1, 1],
                 [1, 1, 1, 1, 1],
                 [-1, -1, -1, -1, 1],
                 [-1, -1, -1, -1, 1],
                 [1, 1, 1, 1, 1]])
zero = np.array([[1, 1, 1, 1, 1],
                 [1, -1, -1, -1, 1],
                 [1, -1, -1, -1, 1],
                 [1, -1, -1, -1, 1],
                 [1, -1, -1, -1, 1],
                 [1, 1, 1, 1, 1]])
def plot_number(number, label='number'):
    plt.figure(figsize=(3, 5))
    plt.imshow(number, cmap=cmap, norm=norm)
    plt.title(label)
    plt.axis('off')
    plt.show()
def save_number_image(number, label, path):

```

```

plt.figure(figsize=(3, 5))
plt.imshow(number, cmap=cmap, norm=norm)
plt.title(label)
plt.axis('off')
plt.savefig(path)
plt.close()
if __name__ == '__main__':
    # plot_number(one, 'one')
    # plot_number(two, 'two')
    # plot_number(three, 'three')
    # plot_number(four, 'four')
    # plot_number(five, 'five')
    # plot_number(six, 'six')
    # plot_number(seven, 'seven')
    # plot_number(eight, 'eight')
    # plot_number(nine, 'nine')
    # plot_number(zero, 'zero')
    save_number_image(one, 'one', '1.png')
    save_number_image(two, 'two', '2.png')
    save_number_image(three, 'three', '3.png')
    save_number_image(four, 'four', '4.png')
    save_number_image(five, 'five', '5.png')
    save_number_image(six, 'six', '6.png')
    save_number_image(seven, 'seven', '7.png')
    save_number_image(eight, 'eight', '8.png')
    save_number_image(nine, 'nine', '9.png')
    save_number_image(zero, 'zero', '0.png')

```

设计的点阵图效果如下：



然后是创建网络。

我们将最简单的 hopfield 模型定义为一个类

```

class Hopfield:
    def __init__(self, number_arrays):
        self.number_arrays = number_arrays
        self.weights = np.zeros((len(number_arrays[0]),
len(number_arrays[0])))
    def print_weights(self):
        print(self.weights)

```

```

def train(self):
    for i in self.number_arrays:
        for j in range(len(i)):
            for k in range(len(i)):
                self.weights[j][k] += i[j] * i[k]
    print(self.weights)
def recall(self, input_array, max_loop=1000):
    output_array = np.zeros(len(input_array))
    tem_array = input_array.copy()
    energy = 0
    loop_count = 0
    while True:
        loop_count += 1
        changed = False
        random_index = [i for i in range(len(tem_array))]
        random_index = shuffle_array(random_index)
        print(random_index)
        for loop in range(len(tem_array)):
            energy = 0
            i = random_index[loop]
            for j in range(len(tem_array)):
                energy += self.weights[i][j] * tem_array[j]
            if energy > 0:
                if tem_array[i] != 1:
                    changed = True
                    print("changed", i)
                else:
                    changed = changed
                    tem_array[i] = 1
            elif energy < 0:
                if tem_array[i] != -1:
                    changed = True
                    print("changed", i)
                else:
                    changed = changed
                    tem_array[i] = -1
        # plot_number(one_dim_array_to_two_dim(tem_array),
'tem_array')
        if not changed:
            output_array = tem_array
            break
        if loop_count > max_loop:
            break

```

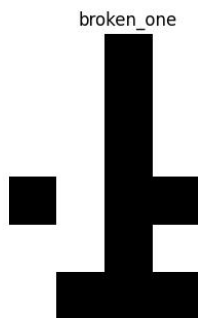
```
return output_array
```

这里使用的是最简单的 hopfield 算法，其中 max_loop 用于防止反复循环无法收敛。

接下来是完成测试工作，首先我们再写一个随机让原数字点阵失真的函数：

```
def random_broke_number(number, broke_count = -1):  
    if broke_count == -1:  
        broke_count = np.random.randint(0, len(number) / 2)  
    for i in range(broke_count):  
        index = np.random.randint(0, len(number))  
        number[index] = -1 if number[index] == 1 else 1  
    return number
```

以数字 1 为例，我们可以生成这样的点阵：

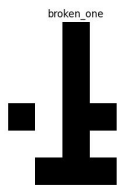


其余的数字均为带有一定坏点的点阵，接下来把图像放入模型进行测试即可。

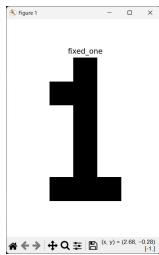
2. 程序实验结果：

此处给出几个坏掉的数字和最终结果的图像

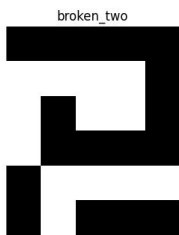
坏掉的 1：



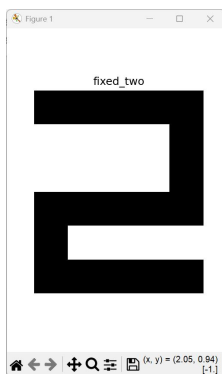
修复后的 1:



坏掉的 2:



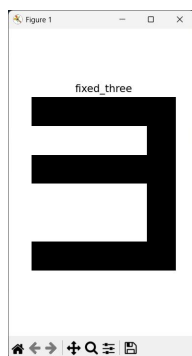
修复的 2:



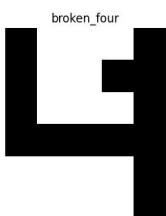
坏掉的 3:



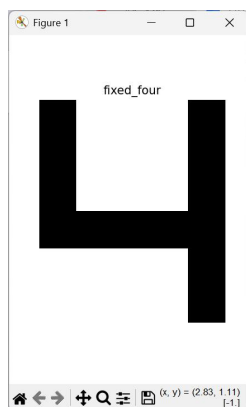
修复的 3:



坏掉的 4:



修复的 4:



3. 分析和改进：

分析实际很简单，上文中用于获取这个结果的模型其实只存了这四个数字，如果不这样，结果根本没法看。原因是，对于 hopfield 模型，人们有这个经验：hopfield 能够存储的模式数量约是神经元的 0.14 倍，也就是说，对于我们这个实验，要求使用 5x6 的点阵，实际上就是 30 个神经元，模型能够正常存储的模式只有 $30 \times 0.14 = 4.2$ 四种，因此，如果我们使用这个神经元数量是无法存储这么多的数字的。此处给出了两种解决方案：

(1)分块存储：很简单的道理，既然一个存不下就多建几个。

(2)扩大神经元数量：也就是让点阵至少拥有 72 个像素即让模型有 72 个神经元，让一个点阵携带的信息更多。

实验报告中的展示的结果使用的是第一种方案。

五 实验小结：

Hopfield 神经网络，结构简单，实现起来容易，计算量也不小，在匹配模式这一块的应用价值很高，同时局限性也很强，尤其是有的模式相似度比较高的时候。但其基于神经科学的思想还是值得学习的。